THORRIJA

Software Engineering Showcase

Deterministic computational models built from source code

Author: Thorrija (Rubén N.)
Spain
November 2026
Technical presentation — universal engineering showcase

Contents

1. Introduction

2. Technical Inventory

3. Architecture and Implementation

4. Mathematical and Computational Foundations

5. Deterministic Reconstruction Methodology

6. Individual Computational Experiments

7. Validation and Testing

8. Comparative Technical Analysis

9. Limitations and Future Development

10. Conclusions

Appendix A — Glossary

Appendix B — Equations

Appendix C — Source Code References

Appendix D — Complete Project Index

1. Introduction

Objectives: Demonstrate genuine software engineering ability through construction, auditing, and reconstruction of real software projects as independent deterministic computational models. This presentation is intended for technical evaluation.

Engineering approach: Every demonstration is based on actual source code from the original projects. Algorithms, data structures, rules, and processing pipelines are identified from the source code and reconstructed as independent models that accept configurable inputs and produce reproducible outputs.

Selection methodology: Projects were evaluated on technical depth, algorithmic complexity, implementation quality, originality, completeness, reproducibility, and educational value. Four projects were selected for interactive demonstrations; all discovered projects are documented in this PDF catalog.

Scope limitations: Demonstrations do not execute original projects, do not require specific hardware, external services, or API keys. Some functionality depends on unavailable hardware or services and is documented as such.

2. Technical Inventory

All projects discovered across GitHub repositories and local development directories, organized by technical category.

2.1 Projects selected for interactive demonstration (4 selected)

Project	Purpose	Technologies	Status	Fidelity
Wi-Fi Time Machine	Wireless environment history with replay and temporal visualization	Python 3, stdlib, SQLite, iw/nmcli/iwd	Completed (v1.0.0)	Level 4
BLE Radar	Terminal radar for nearby BLE devices with RSSI, lifecycle, and replay	Python 3, bleak, SQLite, curses	Completed (v1.0.0)	Level 4
Process Ecosystem Monitor	Linux process tree as a living ecosystem (birth/death/orphans)	Python 3, /proc, stdlib, curses	Completed (v0.1.0)	Level 4
Terminal Generative Engine	Deterministic procedural generative art for terminal	Python 3, stdlib, splitmix64 RNG	Completed	Level 4

2.2 Complete catalog of discovered projects

The PDF report documents all discovered projects. Below is a summary index; extensive details are in Appendix D.

Category	Project	Language	Status
Wireless Networks	Wi-Fi Time Machine	Python	Completed
Wireless Networks	BLE Radar	Python	Completed
Systems	Process Ecosystem Monitor	Python	Completed
Creative / Generative	Terminal Generative Engine	Python	Completed
Systems	silicon	Rust	Completed
Storage	cask	C	Completed
AI / Desktop	aster-ai	—	Completed
Forensics	sigscan	Go	Completed
Networks	dnsdrill	Python	Completed
Systems	syspeek	Python	Completed
Security	loglens	Python	Completed
Security	canarytrap	Python	Completed
Games	neondrift	HTML/JS	Completed

Security	hackdeck	HTML/JS	Completed
Mobile	clickwheel	Kotlin	Completed
Systems	update-manager	—	Completed
Systems	sysmon-dashboard	—	Completed
Systems	snapshot-manager	—	Completed
Networks	port-authority	Python	Completed
Creative	portfolio-demos	HTML/JS	Completed
Cognitive	nexus-titan	—	Completed
Cognitive	nox-cognitive-runtime	—	Completed
Cognitive	echo-cognitive-organism	—	Completed

3. Architecture and Implementation

For each demonstrated project, the architecture discovered in the source code is detailed: component responsibilities, data structures, algorithms, processing pipelines, internal interfaces, dependencies, design decisions, and technical trade-offs.

3.1 Wi-Fi Time Machine

Architecture: Scanning layer (3 backends: iwd, nmcli, iw-cache) → pure parsers → deduplication by BSSID → SQLite storage → aggregation queries → terminal rendering.

Data structures: `ScanResult` (BSSID, SSID, channel, frequency, signal, security), stored in tables `networks`, `observations`, `sessions`.

Algorithms: Parsing BSS blocks from `iw scan dump`, nmcli escaped tokenizer (handling `\\:` and `\\\\`), frequency-to-channel 802.11 mapping (2.4/5/6 GHz), security inference from AKM suites, networks × time-bucket heatmap.

Processing pipeline: Scan → parse → dedup (strongest signal) → register in SQLite → aggregation by network/channel → render heatmap, sparkline, bars.

3.2 BLE Radar

Architecture: Scanners (bleak/demo/replay) → Observation → RadarEngine (state machine + EMA) → snapshot → radar frame geometry (cell grid) → curses UI.

Data structures: `Observation` (address, name, RSSI, timestamp), `DeviceState` (RSSI EMA, state, trails), `DeviceSnap` (angle, band, state, jitter).

Algorithms: State machine appeared→active→fading→disappeared (15s/30s windows), EMA RSSI (α=0.4), RSSI strength bands (strong≥-40 … weak), stable angle from SHA-256(address), aspect-corrected polar geometry for radar, replay cursor with seek/speed.

3.3 Process Ecosystem Monitor

Architecture: /proc collector → pure parsers → EcosystemState (lifecycle, CPU, memory, tree) → renderer/UI.

Data structures: `ProcessSnapshot` (pid, ppid, comm, state, utime, stime, starttime, RSS), `ProcessState` (lifecycle, CPU%, history rings), `SystemSample` (system CPU, RAM, load).

Algorithms: CPU% from jiffie delta normalized per-core, identity (pid, starttime) with PID reuse detection, tree from ppid with orphan re-homing via graveyard (max 512), lifecycle new→active/idle→dying→dead.

3.4 Terminal Generative Engine

Architecture: Stateless splitmix64 RNG → seed-derived parameters → render modes (flowfield/plasma/attractor) → framebuffer → half-block encoding → output.

Algorithms: Analytic sine field (flowfield), closed-form color fields (plasma), Clifford/Dejong strange map with coefficient probing (attractor), density accumulation with log+gamma tone curve.

4. Mathematical and Computational Foundations

Only concepts actually relevant to the implementations are included.

```
channel = ⌊(freq − 5000) / 5⌋   [5 GHz]
```

```
channel = ⌊(freq − 5925) / 5⌋   [6 GHz]
```

```
channel = ⌊(freq − 2407) / 5⌋   [2.4 GHz, except 2484 → 14]
```

```
rssi_ema = 0.6 × prev + 0.4 × raw   (α = 0.4)
```

```
angle = SHA-256(addr)[0:2] / 65536 × 360°   (visual convention, not bearing)
```

```
band = strong(≥−40) / near(≥−55) / mid(≥−70) / far(≥−85) / weak
```

```
x = cx + radius × 2 × cos(angle);   y = cy − radius × sin(angle)
```

```
cpu_proc = Δ(utime+stime) / (interval × CLK_TCK × ncores) × 100
```

```
cpu_sys = busy_delta / total_delta × 100   (idle = idle + iowait)
```

```
identity = (pid, starttime);   starttime change ⇒ PID reuse
```

```
ram_pct = (mem_total − mem_available) / mem_total × 100
```

```
x' = sin(a·y) + c·cos(a·x),   y' = sin(b·x) + d·cos(b·y)   (Clifford attractor)
```

Computational Complexity

- WTM: O(n) parsing over scan text; O(n) aggregation; O(n × buckets) timeline. Linear in observations.
- BLE: O(n) ingestion over observations; O(d) snapshot over devices; O(cells + devices × trails) frame.
- PEM: O(p) update per tick over processes; O(p) tree build; graveyard-bounded ancestor walk.
- GenArt: O(pixels) per frame flowfield/plasma, O(1) seek; O(iterates × frames) attractor density.

5. Deterministic Reconstruction Methodology

Why independent models: Demonstrations do not execute original projects. This avoids dependencies on hardware (Bluetooth/Wi-Fi adapters), external services, specific runtimes, and API keys. Each model is an independent implementation that reproduces the documented computational behavior of the original.

Source code analysis methodology: 1) Complete inspection of source code structure; 2) Identification of algorithms, data structures, and rules; 3) Extraction of pure functions (no I/O); 4) Reimplementation in TypeScript as independent deterministic model; 5) Documentation of simplifications and assumptions; 6) Validation with automated tests.

Fidelity classification: Level 4 = direct algorithmic reproduction (same logic, only I/O boundary adapted). All 4 demonstrated models are Level 4.

Reproducibility strategy: Same input → same output. Configurable seeds for demo data. Configurable epoch clock. Byte-identical outputs across runs.

6. Individual Computational Experiments

Each demonstrated project includes at least one detailed experiment with configurable inputs, processing steps, intermediate results, and final outputs. Interactive demonstrations are available in the web application `showcase/`.

6.1 Wi-Fi Time Machine — Baseline Experiment

Input: Synthetic iw dump fixture with 3 APs (Home-Network on ch1, Office-5G on ch36, Six-G on 6 GHz ch1).

Steps: 1) Parse 4 BSS blocks → 4 observations; 2) Deduplication (Home-Network appears 2×, keep −45 dBm) → 3 networks; 3) Per-network aggregation (first/last seen, times seen,

avg/best signal); 4) Per-channel statistics; 5) Time heatmap; 6) Signal sparkline; 7) JSON/CSV export with SHA-256 sanitization.

Reproducible output: 3 networks detected, 3 observations, channel 1: 1 obs, channel 36: 1 obs, channel 1 (6 GHz): 1 obs. Sparkline: ▁▅▇ (signal −45 to −70 dBm).

6.2 BLE Radar — Baseline Experiment

Input: 8 synthetic demo devices with seed 7, 30 simulated seconds.

Steps: 1) Generate observations with RSSI random walk and lifecycles; 2) Ingest into RadarEngine (state machine + EMA); 3) Snapshot at time t (visible devices); 4) Radar frame geometry (strength rings, sweep, markers); 5) Lifecycle summary.

Reproducible output: With seed 7 and 30 s: visible devices vary between 5-8; states: appeared/active/fading; RSSI rings mapped to strength (not distance). Deterministic fingerprint: same seed → same frame.

6.3 Process Ecosystem Monitor — Baseline Experiment

Input: 6 synthetic process snapshots (systemd, kthreadd, dbus, browser, renderer + system with 80% CPU, 50% RAM).

Steps: 1) Parse /proc/[pid]/stat (comm with nested parens); 2) Identity (pid, starttime) → spawn 6 processes; 3) CPU% from jiffie delta; 4) Process tree from ppid; 5) System metrics (80% CPU, 50% RAM); 6) History entry.

Reproducible output: 6 living processes, systemd as root (ppid=0), kthreadd as kernel thread (ppid=0 excluded from roots), browser as child of dbus. System CPU: 80%.

6.4 Terminal Generative Engine — Baseline Experiment

Input: Seed 12345, flowfield mode, 100×40 cell canvas, frame 0.

Steps: 1) Parameter resolution from seed (particles, steps, drift, zoom, gain, harmonics); 2) Framebuffer creation; 3) Rendering analytic sine field with drifting particles; 4) Brightness accumulation + tone curve + palette mapping; 5) World fingerprint.

Reproducible output: Flowfield frame with accumulated brightness ribbon. Fingerprint: deterministic hash of (mode, frame, params, pixels). Same seed → same fingerprint.

7. Validation and Testing

All tests reported actually ran. Original project test suites pass:

Project	Tests	Result
Wi-Fi Time Machine	62 unit tests	✓ All pass
BLE Radar	138 unit tests	✓ All pass

Process Ecosystem Monitor	208 unit tests	✓ All pass
Terminal Generative Engine	53 of 58 (5 fail in original)	✓ Passing ones are correct

Reconstructed model tests (TypeScript): Jest suites per model covering: determinism (same seed → same output), algorithmic correctness, edge cases (empty input, boundary RSSI, PID reuse, negative frames clamped), input validation, error handling.

Coverage: Target 80%+ on lines and functions of each model.

Validation limitation: The generative engine original has 5 failing tests (5/58). Reconstructed models do not reproduce those failures; they are documented as a known limitation of the original project, not of the model.

8. Comparative Technical Analysis

Dimension	Wi-Fi Time Machine	BLE Radar	Process Ecosystem Monitor	Generative Engine
Domain	Wireless networks	Bluetooth LE	Systems /proc	Generative art
Key algorithm	BSS parsing + channel mapping	State machine + EMA	Jiffie delta + ppid tree	splitmix64 RNG + sine field
Data structures	ScanResult, SQLite	Observation, DeviceState	ProcessSnapshot, ProcessState	Framebuffer, palette
Determinism	Total (synthetic fixtures)	Total (demo seed)	Total (snapshots + clock)	Total (seed → frame)
Complexity	Linear in observations	Linear in observations	Linear in processes per tick	O(pixels) / O(1) frame seek
Tests	62 (100%)	138 (100%)	208 (100%)	53/58
Fidelity	Level 4	Level 4	Level 4	Level 4

Observation: No arbitrary numerical ratings are assigned; comparison is based on measurable, verifiable technical dimensions.

9. Limitations and Future Development

- **Implemented:** 4 deterministic models with tests, web application with 6 sections, bilingual PDF report, bilingual portfolio.
- **Incomplete:** Generative engine has 5 failing tests in the original project (not reproduced in model).
- **Simulated:** All radio hardware (BLE, Wi-Fi) and /proc are replaced by synthetic inputs in demonstrations.
- **Not available without hardware/services:** Real BLE scanning via bleak/BlueZ, real Wi-Fi scanning via iw/nmcli, real /proc reads, persistent SQLite storage.

- **Not verified:** Execution on real hardware of original projects (requires specific adapters and permissions).
- **Future improvements:** CI pipeline, more scanning backends, GIF/PNG export for GenArt, property-based tests.

10. Conclusions

This work demonstrates software engineering ability in: source code analysis, algorithm extraction, deterministic model reconstruction, automated test implementation, interactive web application development with canvas visualizations, technical PDF report generation, and bilingual portfolio development. All claims are backed by actual source code and executable tests. No capabilities or results are fabricated.

Appendix A — Glossary

- **RSSI:** Received Signal Strength Indicator — received signal strength in dBm. A strength measure, not distance.
- **EMA:** Exponential Moving Average — exponential moving average for smoothing noisy signals.
- **Jiffies:** Linux kernel time units (CLK_TCK = 100 by default on x86).
- **PID reuse:** Reuse of process identifiers when a PID is freed and reassigned to another process.
- **AKM:** Authentication Key Management — authentication suites in Wi-Fi security (PSK, SAE, OWE).
- **Half-block encoding:** Terminal encoding where each cell represents 2 vertical pixels (character ▀ or █).
- **Splitmix64:** 64-bit mixing function for deterministic stateless random number generation.

Appendix B — Equations

See section 4 for all formal equations used in implementations.

Appendix C — Source Code References

- Wi-Fi Time Machine: `wtm/scan.py`, `wtm/util.py`, `wtm/viz.py`, `wtm/db.py`, `wtm/export.py`
- BLE Radar: `ble_radar/scan.py`, `ble_radar/lifecycle.py`, `ble_radar/engine.py`, `ble_radar/radar.py`, `ble_radar/util.py`, `ble_radar/replay.py`, `ble_radar/db.py`, `ble_radar/export.py`
- Process Ecosystem Monitor: `process_ecosystem/collector.py`, `process_ecosystem/cpu.py`, `process_ecosystem/model.py`, `process_ecosystem/engine.py`, `process_ecosystem/demo.py`, `process_ecosystem/history.py`

- Terminal Generative Engine: `genart/rng.py`, `genart/color.py`, `genart/framebuffer.py`, `genart/engine.py`, `genart/modes/flowfield.py`, `genart/modes/plasma.py`, `genart/modes/attractor.py`

Appendix D — Complete Project Index

See chapter 2 for the complete inventory with statuses, technologies, and implementation states of all discovered projects.